

Estudio de Factibilidad Técnica y Arquitectura del Lenguaje de Programación Astra

Análisis de Factibilidad y Conflictos Técnicos

Trinomio ARC/ORC, Hilos Verdes y FFI Zero-Copy

La integración simultánea de un modelo de gestión de memoria basado en conteo automático de referencias con recolector de ciclos (ARC/ORC), un entorno de ejecución concurrente basado en hilos verdes (fibras) y una interfaz de funciones foráneas (FFI) de cero copia expone fricciones arquitectónicas fundamentales en la teoría de sistemas de cómputo¹.

Fragmento de código

// Ejemplo de interfaz FFI y paso de buffer en Astra

```
import std::ffi::{CBuffer, PyBuffer}
```

```
pub fn process_matrix_ffi(py_arr: PyBuffer[f64]) -> Result<(), FfiError> {
    unsafe {
        val raw_ptr: *const f64 = py_arr.data_ptr()
        val length: usize = py_arr.len()

        // Llamada directa a C-ABI sin duplicación de memoria
        c_lapack_dgemm(raw_ptr, length)
    }
    Ok(())
}
```

El primer punto de conflicto radica en la atomicidad del contador de referencias dentro de un planificador de hilos verdes con robo de trabajo (*work-stealing scheduler*)¹. Si un hilo verde

migra entre diferentes hilos del sistema operativo (hilos OS o hilos M), los incrementos y decrementos del contador de referencias (RC) requieren instrucciones atómicas de bus de memoria (LOCK XADD en x86_64 o LDADD en AArch64). La sobrecarga de la atomicidad destruye la ventaja competitiva de rendimiento frente a un recolector de basura rastreador (*tracing GC*)¹. Para mitigar este impacto, Astra debe implementar un esquema de **RC No Atómico Local a la Fibra** respaldado por transferencias de propiedad estricta (*ownership move*) o aislamiento de grafos de objetos (*isolated object graphs*), similar a la semántica de

Nim o Yo¹. Cuando una estructura de datos debe compartirse entre fibras ejecutadas en diferentes núcleos, la arquitectura debe forzar la promoción del objeto a un contenedor atómico explícito (AtomicRef[T]) o realizar una clonación profunda (*deep copy*)¹.

El segundo conflicto crítico se manifiesta en la coexistencia entre FFI Cero Copia y la liberación determinista de memoria por ARC³. En un intercambio de arreglos NumPy o PyTorch mediante el protocolo PyBuffer, Astra obtiene un puntero crudo a la memoria gestionada por la máquina virtual de Python⁷. Si la fibra de Astra que consume este buffer finaliza su alcance (*scope*) y libera el mango (*handle*) local, la estructura de control de Astra no debe invocar el deasignador de la memoria mientras la librería de C o Python mantenga lecturas/escrituras activas.

Recíprocamente, si Astra pasa un buffer de su propia memoria heap gestionada por ARC a una función externa de C, el recolector ORC podría liberar la memoria a mitad de la ejecución foránea si detecta que la variable no tiene consumidores visibles dentro del código escrito en Astra. Para garantizar la seguridad sin recurrir a la copia de datos, el compilador debe inyectar **anclas de alcance** (*scope tethers*) y registrar incrementos de referencia en el contador del objeto foráneo mediante Py_INCREF o fijado de punteros (*pointer pinning*) durante todo el tiempo de vida de la llamada FFI⁷.

Cambio de Contexto en FFI Sincrónico Bloqueante

El uso de un planificador de hilos verdes implica mapear N fibras sobre M hilos del sistema operativo mediante P procesadores lógicos ($N : M : P$), similar al modelo de ejecutor de Go². La llamada sincrónica a una biblioteca de C o a una extensión de Python bloqueante (por ejemplo, cálculos matriciales pesados en C/Fortran o llamadas con adquisición del GIL de Python) introduce la parálisis del hilo M subyacente².

Cuando la Fibra A invoca una función FFI bloqueante, el planificador no puede realizar un cambio de contexto cooperativo dentro del espacio de usuario porque el código de C/Python no contiene puntos de ceder paso (*yield points*) compilados por Astra². Si el hilo M_1 se bloquea en la llamada de C, el procesador lógico P_1 asignado a ese hilo queda inactivo, dejando sin ejecutar las restantes fibras en la cola local².

Para solventar esta problemática sin abandonar la sintaxis sincrónica, el entorno de ejecución de Astra implementa el siguiente mecanismo de desacoplamiento de hilos basado en el patrón `entersyscall / exitsyscall` de Go²:

- **Pre-Invocación FFI (`entersyscall`):** Antes de cruzar la frontera C-ABI, la fibra notifica al runtime. El planificador desvincula el procesador lógico P_1 del hilo actual M_1 , dejando a M_1 como un hilo dedicado exclusivamente a la llamada foránea².
- **Transferencia de Estado y Continuidad:** El procesador P_1 es asumido por un nuevo hilo del sistema operativo (M_2) extraído del pozo de hilos (*thread pool*), permitiendo

que la ejecución de las fibras subsecuentes continúe sin latencia agregada².

- **Post-Invocación FFI (exitsyscall):** Al regresar de la llamada de C, el hilo M_1 intenta reasociarse a un procesador P libre². Si no hay procesadores disponibles, la FibrA se suspende en la cola global de listos y el hilo M_1 entra en reposo en el pozo del runtime².

Este proceso resuelve el bloqueo pero introduce un costo de cambio de contexto del sistema operativo (~1 a 3 microsegundos) y un posible incremento en las fallas de caché L1/L2. Para mitigar esta penalización en llamadas FFI ultrarrápidas, Astra permite anotaciones explícitas de atributos en las declaraciones FFI:

- @ffi::direct: Indica que la función de C es ultra-corta (<50ns) y no bloqueante. Se ejecuta directamente en el stack de la fibra sin desvincular el procesador P .
- @ffi::blocking: Aplica la desvinculación completa del hilo M mediante entersyscall.

Gestión de Referencias Cíclicas en ARC/ORC

El conteo de referencias puro falla ante la presencia de estructuras de datos cíclicas (por ejemplo, grafos dirigidos o listas doblemente enlazadas), provocando fugas de memoria (*memory leaks*) invisibles⁴. Para resolver esto sin implementar un recolector de rastreo global (*Stop-The-World*), Astra adopta el algoritmo de **Eliminación de Prueba** (*Trial Deletion*) derivado del trabajo de Bacon-Rajan, optimizado por el runtime ORC¹.

El algoritmo opera sobre la premisa de que un ciclo de basura solo se crea cuando el contador de referencias de un objeto se decrementa a un valor mayor que cero ($RC > 0$)¹. El ciclo de recolección local se divide en cuatro estados principales mediante un esquema de coloración de nodos en el grafo de objetos¹:

Color del Nodo	Estado del Objeto	Significado Semántico en ORC
Negro (<i>Black</i>)	Vivo / Estable	El objeto está en uso activo o ha sido verificado como alcanzable desde la raíz ¹ .
Púrpura (<i>Purple</i>)	Sospechoso de Ciclo	Objeto cuyo RC fue decrementado pero permaneció > 0 . Se añade al buffer de raíces potenciales (possible_roots) ¹ .

Gris (<i>Gray</i>)	Marcado en Prueba	Se ha decrementado provisionalmente el RC interno mediante el recorrido de sus aristas salientes ¹ .
Blanco (<i>White</i>)	Basura Cíclica	Verificado como inalcanzable externamente ($RC =$ tras decremento de prueba). Candidato a liberación ¹ .

Algoritmo de Eliminación de Prueba (Bacon-Rajan en ORC)

El proceso de recolección incremental se formaliza de forma local en cada hilo/fibra mediante las siguientes fases¹:

1. **Fase de Marcado Gris (MarkGray)**: Para cada nodo $v \in \text{possible_roots}$, el algoritmo realiza un recorrido en profundidad (*DFS*). Por cada arista saliente $v \rightarrow w$, se decrementa temporalmente el recuento de referencias del nodo destino ($RC(w) \leftarrow RC(w) - 1$) y se colorea w de gris¹. Este proceso elimina los recuentos de referencias generados internamente dentro del ciclo de la estructura de datos¹.
2. **Fase de Escaneo (Scan)**: Se examinan los nodos procesados¹. Si un nodo gris w mantiene un recuento $RC(w) > 0$, indica que existen referencias externas fuera del subgrafo cíclico bajo prueba¹. En este caso, se ejecuta `ScanBlack(w)`, revirtiendo los decrementos de prueba y restaurando el color negro a todo el subgrafo alcanzable desde w ¹. Si $RC(w) = 0$, el nodo se colorea de blanco, confirmándose como basura cíclica aislada¹.
3. **Fase de Recolección Blanco (CollectWhite)**: Todos los nodos blancos son desasignados determinísticamente¹. Sus destructores son invocados en un pase de liberación de dos etapas para evitar el acceso a memoria ya liberada por parte de nodos hermanos¹.

Fragmento de código

```
// Estructura de datos que demuestra la gestión del recolector ORC
```

```

class GraphNode {
    pub val id: u64
    pub var neighbors: List[GraphNode] = List::new()
}

pub fn create_cycle() {
    val a = GraphNode(id: 1)
    val b = GraphNode(id: 2)

    a.neighbors.append(b)
    b.neighbors.append(a) // Se forma un ciclo (a <-> b)

    // Al salir de la función, 'a' y 'b' pierden sus variables de stack.
    // El decremento de RC deja RC(a) = 1 y RC(b) = 1.
    // ORC añade 'a' y 'b' al buffer 'possible_roots' (Color Púrpura).
    // El recolector incremental ejecuta Trial Deletion y libera el ciclo sin STW.
}

```

Puntos Ciegos y Aspectos No Analizados

Propagación de Excepciones y Pánicos en la Frontera FFI

Uno de los principales vacíos en la especificación de lenguajes de alto rendimiento es el comportamiento de las interrupciones no controladas a través de fronteras ABI¹⁵. El desenrollado de la pila (*stack unwinding*) de Astra no debe cruzar una frontera de código C, ya que el ABI de C no define tablas de desempaqueado de marcos de pila (*unwind tables*)¹⁵. Si una función de C o C++ invoca un *panic* o lanza una excepción `std::exception` mientras es ejecutada dentro de una fibra de Astra, el resultado estándar es un comportamiento indefinido (*Undefined Behavior*) o el colapso inmediato del proceso (SIGSEGV / SIGABRT)¹⁵.

Para salvaguardar la estabilidad, Astra implementa una arquitectura de **Barreras ABI Estrictas** estructurada en tres niveles de protección:

- **Pánicos de Astra hacia C:** Ninguna función exportada con el atributo `extern "C"` puede propagar un pánico. El compilador inyecta un bloque autogenerado `catch_unwind` alrededor del cuerpo de la función. Si la función entra en pánico, el marco captura el evento, convierte el error en un código de estado numérico de C o aborta de forma segura el proceso imprimiendo la traza.
- **Excepciones de C++ hacia Astra:** Las funciones de C++ enlazadas deben envolverse en un adaptador `extern "C-unwind"` capaz de capturar la excepción en el lado de C++ y transformarla en una estructura de error explícita `Result[T, FfiError]` consumible por Astra¹⁵.
- **Manejo de Fallos de Segmentación Foráneos:** Si la biblioteca C ejecuta una desreferenciación nula o violación de acceso (SIGSEGV, SIGFPE), el runtime de Astra, mediante manejadores de señales dedicados (`sigaction` en POSIX o `Vectored Exception`

Handling en Windows), intercepta la señal en el hilo M activo. El runtime traduce la señal en un Pánico de Fibra aislado, permitiendo abortar únicamente la fibra responsable sin derribar todo el clúster de cómputo.

Modelo de Consistencia de Memoria y Prevención de Data Races

La presencia de hilos verdes ejecutados en paralelo sobre múltiples núcleos exige definir de forma rigurosa un Modelo de Memoria (*Memory Consistency Model*). Astra adopta un modelo de **Secuencia Consistente para Programas Libres de Carreras de Datos** (SC-DRF, *Sequentially Consistent for Data-Race-Free*).

Para prevenir *data races* sin sacrificar el rendimiento científico, Astra impone tres reglas estructurales en su sistema de tipos y compilador:

1. **Inmutabilidad por Defecto:** Las variables definidas con la palabra clave `val` son inmutables a nivel profundo (*deeply immutable*). Pueden compartirse entre múltiples fibras de forma libre y simultánea sin necesidad de sincronización explícita o barreras de memoria¹⁸.
2. **Semántica de Transferencia de Propiedad (*Move Semantics*):** Las variables mutables (`var`) pertenecen a un único hilo de ejecución. Para pasar un objeto mutable `T` a otra fibra mediante el canal de comunicación o invocación `spawn`, se requiere la instrucción de consumo explícito `transfer(var)`. Esta operación invalida la variable en la fibra emisora, garantizando que jamás existan dos accesos mutables simultáneos sobre la misma dirección de memoria⁴.
3. **Barreras de Memoria e Instrucciones Atómicas:** Cuando se requiere la modificación concurrente de variables contiguas en memoria (por ejemplo, arreglos multidimensionales compartidos en cálculos numéricos), el acceso debe realizarse a través de primitivas explícitas `Atomic[T]` que inyectan barreras de memoria a nivel de microarquitectura (*Acquire-Release semantics*):

Escritura: `AtomicStore(x, v, Release)` $\implies$ Garantiza la visibilidad previa de escrituras en

Lectura: `AtomicLoad(x, Acquire)` $\implies$ Garantiza que lecturas posteriores no se reordenen ante

Estabilidad de la ABI y Monomorfización

La arquitectura propone eludir la monomorfización de funciones genéricas durante la fase de desarrollo para mantener tiempos de compilación instantáneos (<10ms), utilizando en su lugar despacho dinámico mediante tablas de `vtables` o tablas de testigos de protocolos (*Protocol Witness Tables*, PWT) similares al compilador SIL de Swift¹⁹.

Impacto Exacto de Rendimiento

El uso de *Protocol Witness Tables* en modo desarrollo implica empaquetar los tipos concretos en contenedores existenciales (*Existential Containers*), los cuales contienen un búfer en línea

para valores pequeños (≤ 24 bytes) o un puntero al heap para valores mayores, junto con un puntero a la *Value Witness Table* (VWT) para la gestión del ciclo de vida y un puntero a la *Protocol Witness Table* (PWT) para el despacho de métodos¹⁹.

Esta indirección de punteros introduce costos de rendimiento computacional:

- **Indirección en llamadas:** Desreferenciación doble de puntero por cada invocación de método genérico, eliminando las posibilidades de *inlining* e inoculando fallas en la predicción de saltos de la CPU (*branch misprediction*)¹⁹.
- **Presión sobre la caché de instrucciones L1:** Incapacidad de vectorizar bucles numéricos mediante instrucciones SIMD (AVX-512 / NEON) debido a la naturaleza dinámica de los tipos. El código ejecutado mediante VTables registra una degradación de rendimiento de entre un **15% y un 350%** en algoritmos numéricos de cómputo intensivo con respecto al código monomorfizado.

Estabilidad de la ABI Binaria Dinámica

En modo producción, la monomorfización genera copias de funciones para cada tipo concreto sustituido (`process_f64`, `process_i32`), lo que conduce al fenómeno de la inflación de binarios (*code bloat*). Sin embargo, la monomorfización rompe la estabilidad de la ABI en bibliotecas dinámicas compartidas (`.so` / `.dylib`), ya que las funciones genéricas expuestas no poseen una dirección de entrada fija ni un diseño de pila predecible sin conocer el parámetro de tipo.

Para solucionar esta divergencia, Astra adopta una política de compilación dual:

- **Bibliotecas Dinámicas Expuestas:** Utilizan un ABI estable basado en *Protocol Witness Tables*, preservando la compatibilidad binaria inter-versión.
- **Binarios Finales y Módulos Internos:** El compilador aplica *Monomorfización Codiciosa* (*Greedy Monomorphization*) combinada con *Devirtualización* en el backend LLVM, inlinando las llamadas a VTables cuando el tipo concreto se deduce estáticamente en el grafo de llamadas del programa.

Análisis Comparativo de Mercado

A continuación se presenta un análisis comparativo de la arquitectura propuesta para Astra en relación con los lenguajes de programación modernos de mayor impacto en sistemas y computación científica:

Lenguaje	Modelo de Memoria Principal	Concurren cy Model	Execution Model	Interopera bilidad FFI	Principal Cuello de Botella o Causa de Fallo Histórico
Astra	ARC/ORC predetermi nado +	Hilos verdes (Fibras) sin async/await	Dual: Intérprete/V M (<10ms) /	Cero copia vía C-ABI y protocolo	Riesgo de complejida d

	Bloques unsafe (Manual/Arenas) ¹	[cite: 2]	AOT C/LLVM ⁴	PyBuffer [cite: 7]	arquitectónica por la integración de múltiples sub-sistemas
Mojo	Ownership/Borrow Checker derivado de Rust + Manual	async/await explícito sobre hilos de SO	AOT vía MLIR / LLVM	Interop directa con runtime de C/Python	Dependencia histórica inicial del runtime de Python y velocidad de maduración del ecosistema
Nim	ARC/ORC con recolector de ciclos (<i>Trial Deletion</i>) ⁴	Hilos de SO / Corrutinas explícitas ⁴	Transpilación a C/C++ y compilación AOT ⁴	Interop directa de velocidad nativa con C/C++ ⁴	Sensibilidad histórica a la regla de case-insensitivity y fragmentación de modelos de GC pasados ⁴
Zig	Gestión explícita/manual (Sin GC, sin RC) ⁴	Hilos de SO (Soporte de hilos verdes removido)	AOT nativo vía LLVM / Backend C propio	Cero sobrecarga con C-ABI (cImport)	Elevada carga mental por gestión manual de memoria y manejo defensivo de asignadores

Julia	Recolector de Basura rastreador (STW Tracing GC)	Hilos de SO cooperativos / Task-based	JIT dinámico vía LLVM con Type Inference	Interop con C/Fortran (ccall) y PyCall	Problema severo de "Time-To-First-Plot" (Latencia de JIT) y pausas de GC no deterministas
Vale	Generational References + Hybrid Generational Memory ²⁴	Concurrencia mediante aislamiento de regiones ⁷	AOT vía LLVM	Fearless FFI con decodificación de memoria ⁷	Sobrecarga de espacio por comprobación de generaciones en punteros y juventud del ecosistema ⁸
Swift	Conteo Automático de Referencias (ARC) atómico ⁶	async/await + Estructura de concurrencia	AOT vía LLVM (Uso extensivo de SIL) ¹⁹	Interop directa C/C++ y Objective-C	Elevada sobrecarga por operaciones atómicas en ARC y peso de meta-datos existenciales ⁶

Lecciones Aprendidas de Proyectos Precedentes

El diseño de Julia demostró la elegancia del despacho múltiple (*multiple dispatch*) para el cómputo científico, pero sufrió importantes barreras de adopción en sistemas de tiempo real y herramientas CLI debido a la latencia de JIT compilation¹⁴. Julia compila el código especializado justo antes de la ejecución de la función cuando se conocen los tipos concretos de los argumentos, provocando retrasos de decenas de segundos al ejecutar simples secuencias de graficado ("Time-to-first-plot")¹⁴. Astra evita este colapso desacoplando el flujo de desarrollo:

utiliza un intérprete/VM o un motor ligero sin optimizaciones de bajo nivel para iteraciones inmediatas (<10ms), posponiendo la compilación intensiva AOT vía C/LLVM únicamente para empaquetado final o pruebas de rendimiento⁴.

Por su parte, Rust demostró la factibilidad de la seguridad de memoria sin recolector de basura mediante un analizador estático de tiempos de vida (*lifetimes*). Sin embargo, la pronunciada curva de aprendizaje y los problemas de diseño con grafos de objetos interconectados o patrones de observadores dificultan la productividad inicial. Astra adopta la flexibilidad de ARC/ORC para que el desarrollador prototipe sin luchar contra el compilador, reservando la verificación explícita de propiedad (*Ownership*) y bloques unsafe para secciones críticas del sistema¹.

Asimismo, la discontinuación del soporte nativo de hilos verdes en Rust (en la versión 0.11) y Zig responde a razones técnicas bien documentadas³. La razón principal fue que abstraer la concurrencia en el runtime introducía una capa no nula sobre las llamadas a funciones FFI de C, forzando el uso de pilas segmentadas (*segmented stacks*) o la asignación de pilas de tamaño fijo excesivo, lo cual violaba el principio de "abstracciones de costo cero"³. Astra asume de forma consciente el costo de un runtime ligero a cambio de erradicar el problema de las "funciones coloreadas" impuesto por el paradigma *async/await*¹².

Recomendaciones y Hoja de Ruta de Arquitectura

Matriz de Trade-Offs de Diseño

Dimensión Arquitectónica	Decisión Propuesta	Impacto en Rendimiento	Complejidad de Implementación	Estrategia de Mitigación Recomendada
Gestión de Memoria	ARC No-Atómico por defecto + Recolector ORC ¹	Bajo overhead en hilos individuales; probable colisión en concurrencia ¹	Alta (Implementación del detector de ciclos <i>Trial Deletion</i>) ¹	Restringir el ARC no atómico a fibras aisladas. Forzar AtomicRef[T] al cruzar fibras.
Concurrencia	Planificador M:N de hilos verdes sin <i>async/await</i> [cite: 2]	Alto rendimiento I/O; latencia incremental en llamadas FFI bloqueantes ²	Muy Alta (Manejo de stacks mutables y cambio de contexto en	Introducir anotaciones @ffi::blocking y @ffi::direct para optimizar rutas de FFI.

			ensamblador)	
Tipado Físico	Unidades de medida verificadas en tiempo de compilación ²⁸	Rendimiento Nulo (Costo Cero). Borrado de tipos en compilación final ²⁸	Media (Algoritmo de Unificación Vectorial de Unidades) ²⁸	Mapear unidades como vectores de exponentes racionales en la IR interna ²⁸ .
Inferencia de Tipos	Inferencia agresiva estática	Nulo en tiempo de ejecución; latencia incremental en fase de compilación	Media-Alta (Algoritmo Hindley-Milner / Constraint-Based)	Limitar la inferencia al ámbito local de funciones; requerir anotaciones en firmas públicas.
Dualidad de Backend	Bytecode VM (Dev) vs. AOT LLVM (Prod) ⁴	Dev: Lento (5x a 10x vs C). Prod: Velocidad Nativa C/C++ ⁴	Muy Alta (Mantenimiento de dos representaciones ejecutables paralelas)	Diseñar un Astra Intermediate Representation (AIR) unificado consumible por ambos backends.

Arquitectura del Compilador Mínimo Viable (MVP)

La arquitectura de la canalización (*pipeline*) del compilador de Astra se organiza secuencialmente en las siguientes etapas operativas:

1. **Analizador Léxico y Sintáctico (*Lexer & Parser*)**: Parser por descenso recursivo escrito a mano con sintaxis limpia basada en indentación o delimitadores simplificados. Genera un Árbol de Sintaxis Concreta (CST) para herramientas IDE (Language Server Protocol) y un Árbol de Sintaxis Abstracta (AST) simplificado.
2. **Comprobador de Tipos y Análisis Dimensional (*Type Checker*)**: Implementa un sistema de inferencia de tipos basado en restricciones y el **Algoritmo de Kennedy** para la comprobación dimensional estática de unidades de medida²⁸.
3. **Transformación a Representación Intermedia (AIR)**: Representación de bajo nivel unificada donde se insertan automáticamente los incrementos y decrementos de referencia (ARC/ORC), los puntos de ceder paso (*yield points*) para el planificador de fibras y el borrado final de los tipos de unidades de medida⁴.

4. Generación de Código Dual:

- **Modo Desarrollo:** Compilación de la AIR a Bytecode para la máquina virtual o emisión rápida de C sin optimizaciones para TCC/Cranefield, garantizando tiempos de inicio inferiores a 10ms.
- **Modo Producción:** Generación de código LLVM IR u optimización AOT C99 con monomorfización completa de genéricos e inlining de funciones numéricas.

Verificación Dimensional en Tiempo de Compilación

Cada tipo numérico parametrizado con una unidad de medida se modela internamente mediante el vector de exponentes del Sistema Internacional (SI)²⁸:

$$\text{Dimensión}(T) = [e_{\text{longitud}}, e_{\text{masa}}, e_{\text{tiempo}}, e_{\text{corriente}}, e_{\text{temperatura}}, e_{\text{sustancia}}, e_{\text{intensidad}}]$$

Fragmento de código

```
// Verificación dimensional en tiempo de compilación (Costo Cero)
```

```
type Meter = f64[1, 0, 0, 0, 0, 0, 0]
```

```
type Second = f64[0, 0, 1, 0, 0, 0, 0]
```

```
type Velocity = f64[1, 0, -1, 0, 0, 0, 0]
```

```
pub fn compute_velocity(dist: Meter, time: Second) -> Velocity {
```

```
    return dist / time // Correcto: [1,0,0...] - [0,0,1...] = [1,0,-1...]
```

```
}
```

```
// Error de Compilación capturado en la fase de Type Checking:
```

```
// val fail: Meter = dist + time // Incompatibilidad de vectores dimensionales
```

Conclusiones

La arquitectura propuesta para Astra es **técnicamente factible**, pero requiere compromisos de diseño en la interacción entre sus módulos:

1. **Gestión de Memoria y Concurrencia:** El soporte de hilos verdes exige que el modelo ARC/ORC sea fundamentalmente **local a la fibra**, evitando instrucciones atómicas en operaciones de referencia cotidianas y requiriendo mecanismos explícitos (AtomicRef o copia de objetos) para la transferencia de datos entre fibras paralelas¹.
2. **Llamadas FFI y Rendimiento:** Para preservar la latencia cero en interop con C/Python, Astra debe clasificar las invocaciones externas en funciones directas o bloqueantes, aislando estas últimas mediante la desvinculación automática del hilo del sistema operativo para prevenir la parálisis del planificador de fibras².

3. **Estrategia de Compilador:** La clave del éxito de Astra reside en la correcta implementación de su **Representación Intermedia (AIR)**, la cual permite la coexistencia del motor de desarrollo de arranque ultrarrápido (<10ms) con la compilación AOT optimizada mediante LLVM para entornos de producción de alto rendimiento.

Fuentes citadas

1. Yo/docs/en-US/CYCLE_COLLECTION.md at develop · shd101wyy/Yo, https://github.com/shd101wyy/Yo/blob/develop/docs/en-US/CYCLE_COLLECTION.md
2. runtime/cgocall.go - The Go Programming Language, <https://go.dev/src/runtime/cgocall.go>
3. Why Green Threads were removed by Rust - Reddit, https://www.reddit.com/r/rust/comments/iyub8b/why_green_threads_were_removed_by_rust/
4. Nim 3.0: A Deep Dive into the Next Generation of Nim - Medium, <https://medium.com/@s.quddin/nim-3-0-a-deep-dive-into-the-next-generation-of-nim-6cd2c92cba5c>
5. Reducing Vale's Memory Management Overhead Through Static, https://www.researchgate.net/publication/355737432_Reducing_Vale's_Memory_Management_Overhead_Through_Static_Analysis
6. ORC – Nim's new memory management - Hacker News, <https://news.ycombinator.com/item?id=25345770>
7. The Most Memory Safe Native Programming Language, <https://vale.dev/memory-safe>
8. Generational References - Evan Ovadia, <https://verdaigon.dev/blog/generational-references-v1>
9. Illustrated Tales of Go Runtime Scheduler. | by Ankur Anand - Medium, https://medium.com/@ankur_anand/illustrated-theses-of-go-runtime-scheduler-74809ef6d19b
10. Internals of Go Scheduler. Process | by Mukesh Pilaniya | Medium, <https://medium.com/@mukeshpilaniya/go-scheduler-768c2246cdec>
11. runtime/cgo: scheduler blocked with goroutines · Issue #8636 - GitHub, <https://github.com/golang/go/issues/8636>
12. Rust chose to drop the green thread library so that it could have no, <https://news.ycombinator.com/item?id=37439886>
13. Go Scheduler - Melatoni, <https://nghiant3223.github.io/2025/04/15/go-scheduler.html>
14. Nim's Memory Management, <https://nim-works.github.io/nimskull/gc.html>
15. Handle Panics When Crossing FFI Boundaries - seaxiang, <https://seaxiang.com/blog/VGuvP5QJ>
16. Panicking (unwinding) across an FFI boundary is very much, <https://news.ycombinator.com/item?id=26876516>
17. FFI - The Rustonomicon - Rust Documentation, <https://doc.rust-lang.org/nomicon/ffi.html>

18. Vale's generational references memory management and Higher, <https://discourse.julialang.org/t/vales-generational-references-memory-management-and-higher-raii-vs-julia-and-other-languages/114457>
19. What's the difference between protocol witness table and vtable in, <https://stackoverflow.com/questions/56794554/whats-the-difference-between-protocol-witness-table-and-vtable-in-swift>
20. Relative Performance of Existential Any - Using Swift, <https://forums.swift.org/t/relative-performance-of-existential-any/77299>
21. Introducing --gc:arc - Nim forum, <https://forum.nim-lang.org/t/5734>
22. Introduction to ARC/ORC in Nim : r/ProgrammingLanguages - Reddit, https://www.reddit.com/r/ProgrammingLanguages/comments/jbkual/introduction_to_arcorc_in_nim/
23. The ARC vs GC Debate : r/ProgrammingLanguages - Reddit, https://www.reddit.com/r/ProgrammingLanguages/comments/1t5apmh/the_arc_vs_gc_debate/
24. Vale's Memory Safety Strategy: Generational References and Regions, <https://verdagon.dev/blog/generational-references>
25. Reducing Vale's Memory Management Overhead Through Static, <https://digitalcommons.calpoly.edu/theses/2348/>
26. The Vale Programming Language, <https://vale.dev/>
27. 0230-remove-runtime - The Rust RFC Book, <https://rust-lang.github.io/rfcs/0230-remove-runtime.html>
28. Conversion of Units of Measurement - UT Austin Computer Science, <https://www.cs.utexas.edu/~novak/units95.html>
29. ts-units - Engineers Tools, <https://engineers.tools/disciplines/computer-engineering/software-engineering/ts-units/>
30. Posit Arithmetic and Dimensional Type Systems, <https://clef-lang.com/docs/design/categorical-foundations/posit-arithmetic-dimensional-type-systems/>
31. Managing Quantities and Units of Measurement in Code Bases, <https://www.intechopen.com/chapters/84425>